

The `==` equality operator compares two object's values, whereas the `is` identity operator compares two object's identities. Chapter 7 covers value and identity. Two objects can store equivalent values, but being two separate objects means they have separate identities. However, whenever you compare a value to `None`, you should almost always use the `is` operator rather than the `==` operator.

In some cases, the expression `spam == None` could evaluate to `True` even when `spam` merely contains `None`. This can happen due to overloading the `==` operator, which Chapter 17 covers in more detail. But `spam is None` will check whether the value in the `spam` variable is literally `None`. Because `None` is the only value of the `NoneType` data type, there is only one `None` object in any Python program. If a variable is set to `None`, the `is None` comparison will always evaluate to `True`. Chapter 17 describes the specifics of overloading the `==` operator, but the following is an example of this behavior:

```
>>> class SomeClass:
...     def __eq__(self, other):
...         if other is None:
...             return True
...
>>> spam = SomeClass()
>>> spam == None
True
>>> spam is None
False
```

The possibility that a class overloads the `==` operator this way is rare, but it's become idiomatic Python to always use `is None` instead of `== None` just in case.

Finally, you shouldn't use the `is` operator with the values `True` and `False`. You can use the `==` equality operator to compare a value with `True` or `False`, such as `spam == True` or `spam == False`. Even more common is to leave out the operator and Boolean value altogether, writing code like `if spam:` or `if not spam:` instead of `if spam == True:` or `if spam == False:`.

Formatting Strings